

SymPy Bug Report

1 Bug 28: solveset returns -1 for a principal cube-root equation

1.1 Minimal Reproducer

```
from sympy import Eq, N, Rational, S, simplify, solveset, symbols

x = symbols("x")
eq = Eq(x**Rational(1, 3), -1)
sol = solveset(eq, x, S.Reals)

print("equation:", eq)
print("solveset:", sol)
for s in sol:
    print("candidate:", s)
    print("lhs at candidate:", eq.lhs.subs(x, s))
    print("residual:", simplify(eq.lhs.subs(x, s) - eq.rhs))
    print("numeric residual:", N(eq.lhs.subs(x, s) - eq.rhs, 30))
print("complex-domain solveset:", solveset(eq, x, S.Complexes))
```

1.2 Observed Behavior

```
equation: Eq(x**(1/3), -1)
solveset: {-1}
candidate: -1
lhs at candidate: (-1)**(1/3)
residual: 1 + (-1)**(1/3)
numeric residual: 1.5 + 0.866025403784438646763723170753*I
complex-domain solveset: EmptySet
```

SymPy returns the finite set $\{-1\}$ for the real-domain call

$$\text{solveset}(\text{Eq}(x^{1/3}, -1), x, \mathbb{R}).$$

However, substituting the returned value into the original SymPy expression does not satisfy the equation. The residual at $x = -1$ is

$$(-1)^{1/3} + 1,$$

which SymPy numerically evaluates as $1.5 + 0.8660254037844386i$, not zero.

1.3 Expected Behavior

The correct result is `EmptySet`. SymPy’s expression `x**Rational(1, 3)` is a principal complex power, not the real-valued cube-root function. Therefore no real x satisfies the principal-power equation $x^{1/3} = -1$ in SymPy’s semantics.

1.4 Mathematical Explanation

For SymPy’s principal complex power,

$$z^{1/3} = \exp\left(\frac{\text{Log}(z)}{3}\right),$$

where `Log` is the principal logarithm. Since $\text{Log}(-1) = i\pi$, the principal cube root of -1 is

$$(-1)^{1/3} = \exp(i\pi/3) = \frac{1}{2} + \frac{\sqrt{3}}{2}i.$$

Thus

$$(-1)^{1/3} \neq -1 \quad \text{and} \quad (-1)^{1/3} + 1 \neq 0.$$

The returned value is not merely an alternate branch representation of the same solution. It is the value produced by the real cube-root function $\text{real_root}(-1, 3) = -1$, but the input equation was written using SymPy’s principal `Pow`. Solving the principal-power equation therefore requires respecting the principal branch, and -1 is an extraneous solution.

1.5 Source-Level Diagnosis

The diagnosis identifies the responsible behavior in `sympy/solvers/solveset.py`, in the function `_invert_real`, around line 284 in the inspected SymPy 1.14.0 source. The public call enters `solveset`; because the requested domain is real, the symbol is recast to a real dummy and the equation is passed to `_solveset` with checking enabled. `_solveset` rewrites the equation as `x**(1/3) + 1` and calls `_invert`, which dispatches to `_invert_real` for the real-domain inversion of the rational power.

The key rule identified in `_invert_real` is the odd-denominator rational-power branch:

```
if den % 2 == 1:
    root = Lambda(n, real_root(n, expo))
    res = imageset(root, g_ys)
    ...
    return _invert_real(base, res, symbol)
```

For $x^{1/3} + 1 = 0$, this branch applies `real_root` to the target set $\{-1\}$, producing $\{-1\}$ as the preimage for the base x . That transformation is valid for equations explicitly expressed with real-root semantics, but it is not equivalent to inverting SymPy’s principal complex `Pow` on negative real inputs.

The diagnosis also explains why the later checking pass does not remove the extraneous candidate. The finite-solution filtering path uses `domain_check`, which checks whether the expression is finite and defined at the candidate, rather than re-evaluating whether the original equation is true. Since $(-1)^{1/3} + 1$ is finite, the candidate survives even though its residual is nonzero. A plausible fix direction is to restrict this odd-denominator `real_root` inversion branch to expressions whose semantics are actually real-root semantics, or to add a principal-branch guard and equation-truth validation when branch-sensitive inversions have been used.

1.6 Additional Failing Instances

The same failure appears for odd denominators beyond 3. The independent verification cases consider

$$x^{1/n} = -1$$

over the real domain for $n \in \{3, 5, 7, 9, 11, 13\}$. In each case, SymPy's real-domain solver returns $\{-1\}$, while the expected answer is `EmptySet`. The reason is uniform: for every listed odd $n > 1$, the principal value is

$$(-1)^{1/n} = \exp(i\pi/n),$$

which is not -1 . Hence $x = -1$ does not satisfy any of the listed principal-power equations.

Input / Expression	SymPy behavior	Expected behavior
<code>solveset(Eq($x^{1/3}$, -1), x, $\mathbb{R}$)</code>	returns $\{-1\}$	<code>EmptySet</code>
<code>solveset(Eq($x^{1/5}$, -1), x, $\mathbb{R}$)</code>	returns $\{-1\}$	<code>EmptySet</code>
<code>solveset(Eq($x^{1/7}$, -1), x, $\mathbb{R}$)</code>	returns $\{-1\}$	<code>EmptySet</code>
<code>solveset(Eq($x^{1/9}$, -1), x, $\mathbb{R}$)</code>	returns $\{-1\}$	<code>EmptySet</code>
<code>solveset(Eq($x^{1/11}$, -1), x, $\mathbb{R}$)</code>	returns $\{-1\}$	<code>EmptySet</code>
<code>solveset(Eq($x^{1/13}$, -1), x, $\mathbb{R}$)</code>	returns $\{-1\}$	<code>EmptySet</code>

1.7 Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check and classified this as a new instance of a known failure mode. The closest public materials identified were SymPy documentation describing the explicit domain argument to `solveset`, principal-branch behavior for elementary functions, and the distinction between principal powers and real-root interpretations. The available evidence did not identify a public GitHub issue, pull request, Stack Overflow question, mailing-list post, or release-note entry for this exact reproducer returning $\{-1\}$. Although rational powers and principal branches are a known source of solver mistakes, this concrete real-domain equation is previously unreported in the available evidence and remains individually actionable as an issue and as a regression test.

1.8 Proposed SymPy Regression Test

```
import pytest

from sympy import Eq, EmptySet, Rational, S, solveset, symbols

@pytest.mark.parametrize("denominator", [3, 5, 7, 9, 11, 13])
def test_principal_odd_root_equation_has_no_negative_real_solution(denominator):
    x = symbols("x")
    assert solveset(Eq(x**Rational(1, denominator), -1), x, S.Reals) == EmptySet
```